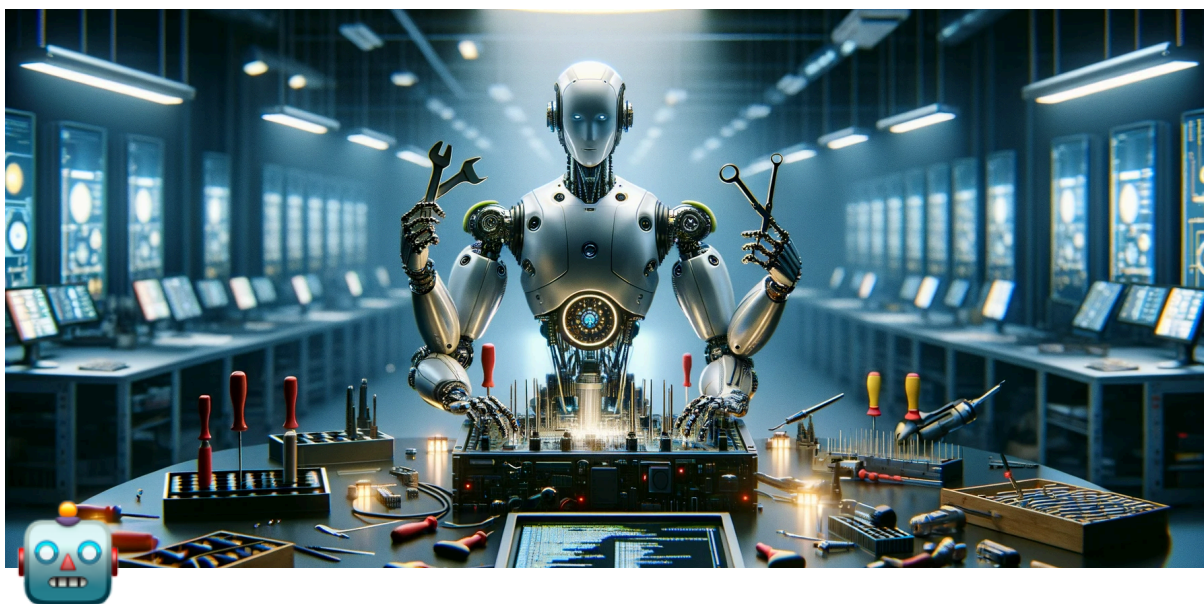




Unidad 7 - Agentes Autónomos y Sistemas Inteligentes

UNR - TUIA - Procesamiento de Lenguaje Natural

Docente teoría: Juan Pablo Manson - jpmanson@gmail.com - [LinkedIn](#)

Diapositivas

https://drive.google.com/file/d/1KMdZGsyHS86FAKnj_2PpjEhuzAzVrWRq/view?usp=sharing

Cuaderno de práctica

Google Colab

 https://colab.research.google.com/drive/1Ror8tAUAWo7Hc-ormVCTZYebpKL06VZ_?usp=sharing



1. Agentes

¿Qué es un Agente?

En su forma más fundamental, un **agente de IA Generativa** puede definirse como una aplicación que intenta **lograr un objetivo** observando el mundo y actuando sobre él utilizando las **herramientas** de las que dispone. Los agentes son **autónomos** y pueden actuar independientemente de la intervención humana, especialmente cuando se les proporcionan objetivos o metas adecuadas que deben alcanzar.

Los agentes también pueden ser **proactivos** en su enfoque para alcanzar sus metas. Incluso en ausencia de conjuntos de instrucciones explícitas de un humano, un agente puede **razonar** sobre lo que debe hacer a continuación para lograr su objetivo final.

Componentes Fundamentales de un Agente

Para comprender el funcionamiento interno de un agente, primero presentemos los componentes fundamentales que impulsan el comportamiento, las acciones y la toma de decisiones del agente. La combinación de estos componentes se puede describir como una **arquitectura cognitiva**, y existen muchas arquitecturas de este tipo que se pueden lograr mezclando y combinando estos componentes.

Centrándose en las funcionalidades principales, existen **tres componentes esenciales** en la arquitectura cognitiva de un agente, como se muestra en la figura:

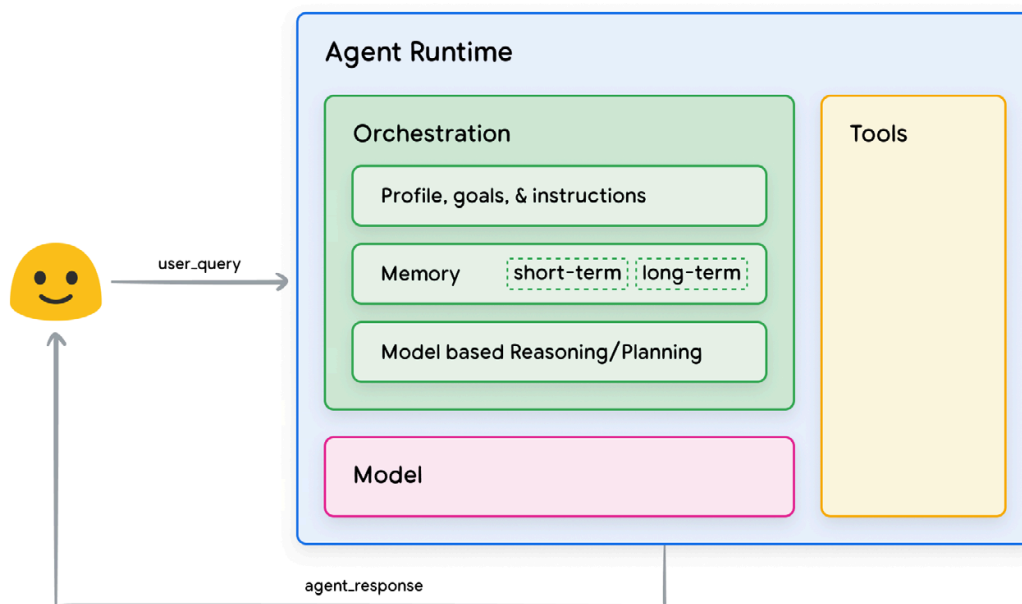


Figura 1. Arquitectura y componentes generales de un agente

El Modelo

En el ámbito de un agente, un **modelo** se refiere al modelo de lenguaje (LM, por sus siglas en inglés) que se utilizará como el **decisor centralizado** para los procesos del agente.

El modelo utilizado por un agente puede ser uno o varios LM de cualquier tamaño (pequeño/grande) que sean capaces de seguir marcos de lógica y razonamiento basados en instrucciones, como **ReAct**, **Chain-of-Thought** o **Tree-of-Thoughts**.

Los modelos pueden ser de propósito general, multimodales o ajustados (*fine-tuned*) según las necesidades de su arquitectura de agente específica. Para obtener los mejores resultados de producción, debe aprovechar un modelo que **mejor se adapte a su aplicación final deseada** e, idealmente, que haya sido entrenado con datos asociados a las **herramientas** que planea utilizar en la arquitectura cognitiva.

Es importante señalar que el modelo **no suele estar entrenado** con la configuración específica del agente (es decir, la selección de herramientas, la orquestación/configuración del razonamiento). Sin embargo, es posible **refinar aún más** el modelo para las tareas del agente proporcionándole ejemplos que muestren las capacidades del agente, incluidas instancias en las que el agente

utiliza herramientas específicas o pasos de razonamiento en diversos contextos.

Las Herramientas

Los modelos fundacionales, a pesar de su impresionante generación de texto e imágenes, siguen limitados por su incapacidad para interactuar con el mundo exterior. Las **Herramientas** cierran esta brecha, capacitando a los agentes para **interactuar con datos y servicios externos** a la vez que desbloquean una gama más amplia de acciones que va más allá de lo que el modelo subyacente puede hacer por sí solo.

Las herramientas pueden adoptar diversas formas y tener diferentes niveles de complejidad, pero generalmente se alinean con los métodos comunes de la API web como **GET, POST, PUT y DELETE**. Por ejemplo, una herramienta podría actualizar la información de un cliente en una base de datos o recuperar datos meteorológicos para influir en una recomendación de viaje que el agente está proporcionando al usuario.

Con las herramientas, los agentes pueden acceder y procesar información del mundo real. Esto les permite admitir sistemas más especializados como la **generación aumentada por recuperación (RAG)**, que extiende significativamente las capacidades de un agente más allá de lo que el modelo fundacional puede lograr por sí mismo. Discutiremos las herramientas con más detalle más adelante, pero lo más importante a comprender es que las herramientas **cierran la brecha entre las capacidades internas del agente y el mundo externo**, desbloqueando una gama más amplia de posibilidades.

La Capa de Orquestación

La **capa de orquestación** describe un proceso cíclico que rige cómo el agente **recibe información, realiza algún razonamiento interno y utiliza ese razonamiento para informar su siguiente acción o decisión**. En general, este bucle continuará hasta que un agente haya alcanzado su objetivo o un punto de detención.

La complejidad de la capa de orquestación puede variar mucho dependiendo del agente y de la tarea que esté realizando. Algunos bucles pueden ser **cálculos simples** con reglas de decisión, mientras que otros pueden contener **lógica encadenada**, involucrar **algoritmos de machine learning adicionales** o implementar otras **técnicas de razonamiento probabilístico**. Discutiremos más

a fondo la implementación detallada de las capas de orquestación de agentes en la sección de arquitectura cognitiva.

Agentes vs. Modelos

Para obtener una comprensión más clara de la distinción entre agentes y modelos, considere la siguiente tabla:

Modelos	Agentes
El conocimiento se limita a lo que está disponible en sus datos de entrenamiento.	El conocimiento se extiende a través de la conexión con sistemas externos a través de herramientas.
Inferencia/predicción única basada en la consulta del usuario. A menos que se implemente explícitamente para el modelo, no hay gestión del historial de sesión o contexto continuo (es decir, historial de chat).	Historial de sesión gestionado (es decir, historial de chat) para permitir la inferencia/predicción de múltiples turnos basada en las consultas del usuario y las decisiones tomadas en la capa de orquestación ⁶ . En este contexto, un "turno" se define como una interacción entre el sistema interactuante y el agente (es decir, 1 evento/consulta entrante y 1 respuesta del agente).
Sin implementación nativa de herramientas.	Las herramientas se implementan de forma nativa en la arquitectura del agente.
No hay una capa de lógica nativa implementada. Los usuarios pueden formular <i>prompts</i> como preguntas simples o usar marcos de razonamiento (CoT, ReAct, etc.) para formar <i>prompts</i> complejos para guiar al modelo en la predicción	Arquitectura cognitiva nativa que utiliza marcos de razonamiento como CoT, ReAct u otros marcos de agente preconstruidos como LangChain.

Arquitecturas Cognitivas: Cómo Operan los Agentes

Imagine a un chef en una cocina concurrida. Su objetivo es crear platos deliciosos para los clientes del restaurante, lo que implica un ciclo de planificación, ejecución y ajuste.

- Reúnen información, como el pedido del cliente y qué ingredientes hay en la despensa y el refrigerador.
- Realizan un **razonamiento interno** sobre qué platos y perfiles de sabor pueden crear basándose en la información que acaban de recopilar.

- Toman **medidas** para crear el plato: picar verduras, mezclar especias, sellar la carne.

En cada etapa del proceso, el chef realiza ajustes según sea necesario, refinando su plan a medida que se agotan los ingredientes o se reciben comentarios de los clientes, y utiliza el conjunto de resultados anteriores para determinar el siguiente plan de acción. Este ciclo de ingesta de información, planificación, ejecución y ajuste describe una **arquitectura cognitiva** única que el chef emplea para alcanzar su objetivo.

Al igual que el chef, los agentes pueden utilizar arquitecturas cognitivas para alcanzar sus objetivos finales procesando información de forma iterativa, tomando decisiones informadas y refinando las siguientes acciones basándose en resultados anteriores. En el centro de las arquitecturas cognitivas de los agentes se encuentra la **capa de orquestación**, responsable de mantener la memoria, el estado, el razonamiento y la planificación. Utiliza el campo de rápida evolución de la ingeniería de *prompts* y los marcos asociados para guiar el razonamiento y la planificación, lo que permite al agente interactuar de forma más efectiva con su entorno y completar tareas.

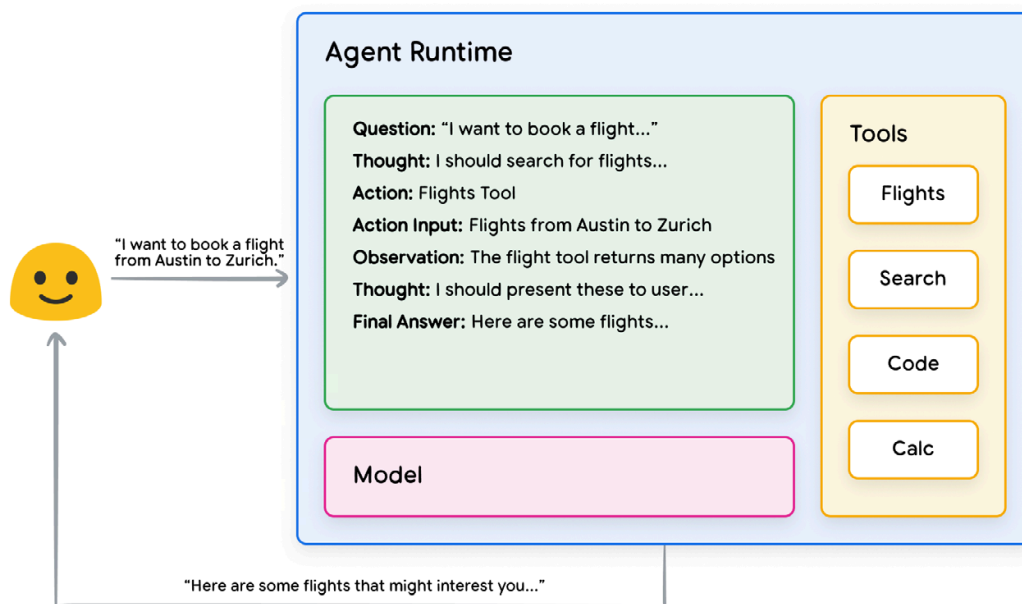
La investigación en el área de los marcos de ingeniería de *prompts* y la planificación de tareas para modelos de lenguaje está evolucionando rápidamente, produciendo una variedad de enfoques prometedores. Si bien no es una lista exhaustiva, estos son algunos de los marcos y técnicas de razonamiento más populares disponibles en el momento de esta publicación:

- **ReAct** (Reasoning and Acting) , un marco de ingeniería de *prompts* que proporciona una estrategia de proceso de pensamiento para que los modelos de lenguaje **razonen** y **actúen** en una consulta de usuario, con o sin ejemplos en contexto. Se ha demostrado que el *prompting* ReAct supera a varias líneas base SOTA y mejora la interoperabilidad humana y la confiabilidad de los LLM.
- **Chain-of-Thought (CoT)** , un marco de ingeniería de *prompts* que permite capacidades de razonamiento a través de pasos intermedios. Hay varias sub-técnicas de CoT, incluyendo auto-consistencia (*self-consistency*), *active-prompt* y CoT multimodal, cada una con fortalezas y debilidades que dependen de la aplicación específica.
- **Tree-of-Thoughts (ToT)**, un marco de ingeniería de *prompts* que es muy adecuado para tareas de exploración o de visión estratégica a futuro (*strategic lookahead tasks*). Generaliza el *prompting* Chain-of-Thought y

permite que el modelo explore varias cadenas de pensamiento que sirven como pasos intermedios para la resolución general de problemas con modelos de lenguaje.

Los agentes pueden utilizar una de las técnicas de razonamiento anteriores, o muchas otras técnicas, para elegir la mejor acción siguiente para la solicitud de usuario dada. Por ejemplo, consideremos un agente que está programado para utilizar el marco **ReAct** para elegir las acciones y herramientas correctas para la consulta del usuario. La secuencia de eventos podría ser algo así:

1. El usuario envía la consulta al agente.
2. El agente comienza la secuencia ReAct.
3. El agente proporciona un *prompt* al modelo, pidiéndole que genere uno de los siguientes pasos ReAct y su resultado correspondiente:
 - a. **Pregunta (Question)**: La pregunta de entrada de la consulta del usuario, proporcionada con el *prompt*.
 - b. **Pensamiento (Thought)**: Los pensamientos del modelo sobre lo que debe hacer a continuación.
 - c. **Acción (Action)**: La decisión del modelo sobre qué acción tomar a continuación.
 - i. Aquí es donde puede ocurrir la elección de la herramienta.
 - ii. Por ejemplo, una acción podría ser una de entre **[Vuelos, Búsqueda, Código, Ninguna]**, donde las 3 primeras representan una herramienta conocida que el modelo puede elegir, y la última representa "ninguna elección de herramienta".
 - d. **Entrada de acción (Action input)**: La decisión del modelo sobre qué entradas proporcionar a la herramienta (si las hay).
 - e. **Observación (Observation)**: El resultado de la secuencia de acción/entrada de acción.
 - i. Este pensamiento/acción/entrada de acción/observación podría repetirse N veces según sea necesario.
 - f. **Respuesta final (Final answer)**: La respuesta final del modelo para proporcionar a la consulta original del usuario.
4. El bucle ReAct concluye y se proporciona una respuesta final al usuario.



Como se muestra en la figura, el modelo, las herramientas y la configuración del agente trabajan juntos para proporcionar una respuesta fundamentada y concisa al usuario basándose en la consulta original del usuario. Si bien el modelo podría haber *adivinado* una respuesta (*hallucinated*) basándose en su conocimiento previo, en su lugar utilizó una herramienta (Vuelos) para buscar información externa en tiempo real. Esta información adicional fue proporcionada al modelo, permitiéndole tomar una decisión más informada basada en datos fácticos reales y resumir esta información al usuario.

En resumen, la calidad de las respuestas del agente puede estar directamente ligada a la capacidad del modelo para razonar y actuar sobre estas diversas tareas, incluida la capacidad de seleccionar las herramientas adecuadas y qué tan bien se ha definido esa herramienta. Al igual que un chef elabora un plato con ingredientes frescos y atento a los comentarios de los clientes, los agentes dependen de un **razonamiento sólido** e **información confiable** para ofrecer resultados óptimos. En la siguiente sección, profundizaremos en las diversas formas en que los agentes se conectan con datos frescos.

Herramientas: Nuestras Claves para el Mundo Exterior

Si bien los modelos de lenguaje destacan en el procesamiento de información, carecen de la capacidad de percibir e influir directamente en el mundo real. Esto limita su utilidad en situaciones que requieren interacción con sistemas o datos externos. Esto significa que, en cierto sentido, un modelo de lenguaje es

tan bueno como lo que ha aprendido de sus datos de entrenamiento. Pero independientemente de la cantidad de datos que le proporcionemos a un modelo, aún carecen de la capacidad fundamental de interactuar con el mundo exterior.

Entonces, ¿cómo podemos capacitar a nuestros modelos para que tengan una interacción en tiempo real y consciente del contexto con sistemas externos?

Las **Funciones, Extensiones, Almacenes de Datos (Data Stores) y Plugins** son formas de proporcionar esta capacidad crítica al modelo.

Si bien reciben muchos nombres, las **herramientas** son lo que crea un vínculo entre nuestros modelos fundacionales y el mundo exterior. Este vínculo con sistemas y datos externos permite a nuestro agente realizar una variedad más amplia de tareas y hacerlo con mayor precisión y confiabilidad. Por ejemplo, las herramientas pueden permitir a los agentes ajustar la configuración del hogar inteligente, actualizar calendarios, obtener información del usuario de una base de datos o enviar correos electrónicos basándose en un conjunto específico de instrucciones.

Las herramientas de Google, entre otras, utilizan tres tipos principales de herramientas con las que los modelos de Google pueden interactuar:

Extensiones, Funciones y Almacenes de Datos (Data Stores). Al equipar a los agentes con herramientas, liberamos un vasto potencial para que no solo comprendan el mundo sino que también actúen sobre él, abriendo puertas a una miríada de nuevas aplicaciones y posibilidades.

Extensiones

La forma más fácil de entender las **Extensiones** es pensar en ellas como un puente que conecta una API con un agente de forma estandarizada, permitiendo a los agentes ejecutar APIs sin problemas, independientemente de su implementación subyacente. Digamos que ha creado un agente con el objetivo de ayudar a los usuarios a reservar vuelos. Sabe que desea utilizar la API de Google Flights para recuperar información de vuelos, pero no está seguro de cómo conseguir que su agente realice llamadas a este *endpoint* de la API.



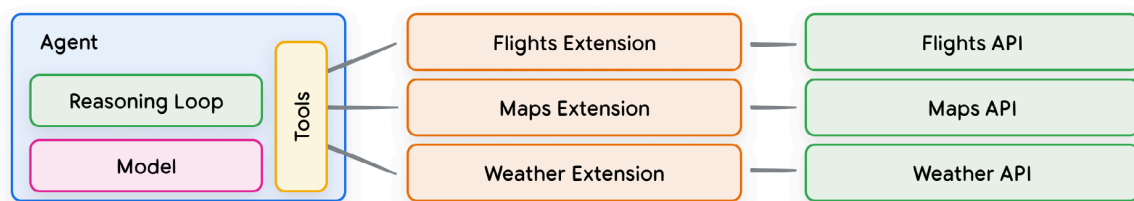
Un enfoque podría ser implementar código personalizado que tome la consulta de usuario entrante, analice la consulta para obtener información relevante y luego realice la llamada a la API. Por ejemplo, en un caso de uso de reserva de vuelos, un usuario podría decir "Quiero reservar un vuelo de Austin a Zúrich". En este escenario, nuestra solución de código personalizado necesitaría extraer "Austin" y "Zúrich" como entidades relevantes de la consulta del usuario antes de intentar realizar la llamada a la API. Pero, ¿qué sucede si el usuario dice "Quiero reservar un vuelo a Zúrich" y nunca proporciona una ciudad de salida? La llamada a la API fallaría sin los datos requeridos y sería necesario implementar más código para detectar casos extremos y casos límite como este. Este enfoque no es escalable y podría fallar fácilmente en cualquier escenario que quede fuera del código personalizado implementado.

Un enfoque más robusto sería utilizar una **Extensión**. Una Extensión cierra la brecha entre un agente y una API al:

1. Enseñar al agente a usar el *endpoint* de la API mediante ejemplos.
2. Enseñar al agente qué argumentos o parámetros se necesitan para llamar con éxito al *endpoint* de la API.



Las Extensiones se pueden diseñar independientemente del agente, pero deben proporcionarse como parte de la configuración del agente. El agente utiliza el modelo y los ejemplos en tiempo de ejecución para decidir qué Extensión, si es que alguna, sería adecuada para resolver la consulta del usuario. Esto pone de relieve una **fortaleza clave de las Extensiones**: sus tipos de ejemplo incorporados, que permiten al agente seleccionar dinámicamente la Extensión más apropiada para la tarea.



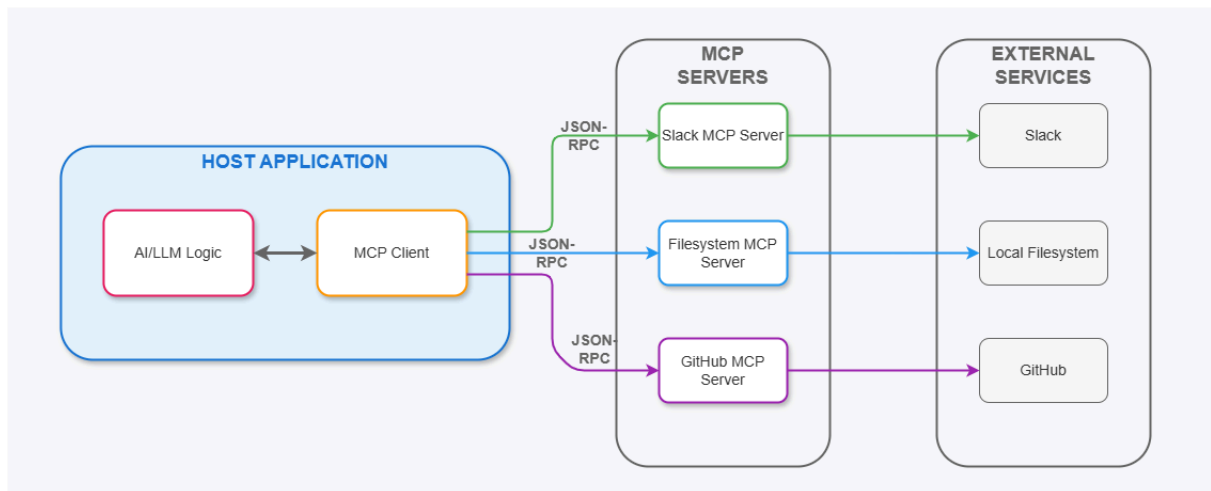
Piense en esto de la misma manera que un desarrollador de software decide qué *endpoints* de API utilizar al resolver y solucionar un problema de un usuario. Si el usuario quiere reservar un vuelo, el desarrollador podría utilizar la API de Google Flights. Si el usuario quiere saber dónde está la cafetería más cercana con respecto a su ubicación, el desarrollador podría utilizar la API de Google Maps. De esta misma manera, la pila agente/modelo utiliza un conjunto de Extensiones conocidas para decidir cuál será la que mejor se adapte a la consulta del usuario.

Model Context Protocol (MCP)

MCP es un estándar abierto diseñado para solucionar el problema de cómo los asistentes de IA y los agentes se conectan a los datos y herramientas externas. Antes de MCP, si querías conectar una IA a Google Drive, Slack o una base de datos, tenías que construir una integración (o extensión) personalizada para cada uno. Era como tener un cargador diferente para cada dispositivo electrónico. MCP funciona como un puerto USB-C universal, ya que proporciona una forma estandarizada para que cualquier IA (el "agente") se conecte y entienda cualquier fuente de datos o herramienta, sin necesidad de integraciones a medida para cada caso.

El protocolo divide la comunicación en tres partes:

1. **MCP Host (El Anfitrión):** Es la aplicación donde vive el agente (por ejemplo, la app de escritorio de Claude o un IDE como Cursor).
2. **MCP Client (El Agente):** El modelo de IA que quiere acceder a información o ejecutar una acción.
3. **MCP Server (El Servidor):** Un pequeño programa "puente" que expone los datos de una herramienta específica (como GitHub, SQLite o Google Drive) en un formato que el agente entiende gracias al protocolo.



Fuente: <https://thelarchitect.com/blog/the-architectural-elegance-of-model-context-protocol-mcp/>

Importancia para los agentes

- **Lectura de Contexto:** Permite al agente "leer" tus archivos locales o bases de datos remotas para tener contexto real antes de responder.
- **Portabilidad:** Si creas un servidor MCP para conectar tu base de datos interna, ese mismo servidor funciona para Claude, ChatGPT, etc., para un agente en Python o cualquier otra herramienta compatible con MCP.
- **Acción:** No solo permite leer datos, sino que habilita a los agentes para ejecutar herramientas (como hacer un commit en Git o consultar una API) de forma segura y controlada.

2. Razonamiento

Quizás una de las tareas más difíciles para un LLM hoy en día es aquella que requiere alguna forma de razonamiento. El razonamiento es una de las áreas más interesantes, debido a los tipos de aplicaciones complejas que pueden surgir de los LLMs.

Ha habido algunas mejoras en tareas que involucran capacidades matemáticas. Dicho esto, es importante señalar que los LLMs actuales luchan por realizar tareas de razonamiento, por lo que esto requiere técnicas de ingeniería de *prompts* aún más avanzadas. Por ahora, cubriremos algunos ejemplos básicos para mostrar capacidades aritméticas.

Prompt:

¿Cuánto es $9.000 * 9.000$?

Salida:

El resultado de multiplicar 9,000 por 9,000 es 81,000,000

Probemos algo más difícil:

Prompt:

Los números impares de este grupo suman un número par: 15, 32, 5, 13, 8
2, 7, 1.

A:

Salida:

No, los números impares de este grupo suman un número impar: 119.

¡Incorrecto! Intentemos mejorar esto mejorando el *prompt*.

Prompt:

Los números impares en este grupo suman un número par: 15, 32, 5, 13, 8
2, 7, 1.

Resuelva dividiendo el problema en pasos. Primero, identifique los números
s impares, añádalos e indique si el resultado es impar o par.

Salida:

Claro, vamos a resolver este problema paso a paso:

Paso 1: Identificar los números impares en el grupo:

Los números impares en el grupo son: 15, 5, 13, 7 y 1.

Paso 2: Sumar los números impares:

$15 + 5 + 13 + 7 + 1 = 41$

Paso 3: Determinar si el resultado es par o impar:

El resultado de la suma es 41, y 41 es un número impar.

Por lo tanto, la suma de los números impares en este grupo es un número impar.

Como podemos ver, si proporcionamos mejores instrucciones combinadas con ejemplos, podría ayudar a obtener resultados más precisos.

Generación de Prompts Zero-Shot

Los grandes modelos de lenguaje como el GPT-3.5 y GPT-4, hoy en día, están ajustados para seguir instrucciones y se entrenan con grandes cantidades de datos, por lo que son capaces de realizar algunas tareas de forma "zero-shot".

Probamos algunos ejemplos de cero disparos en la sección anterior. Aquí hay uno de los ejemplos que utilizamos:

Prompt:

Clasifica el texto en neutral, negativo o positivo.

Texto: Creo que las vacaciones están bien.

Sentimiento:

Salida:

Neutral

Notemos que en el *prompt* anterior no proporcionamos al modelo ningún ejemplo de texto junto con sus clasificaciones, el LLM ya entiende "sentimiento" -- eso es la capacidad de zero-shot en acción.

La sintonización fina (fine tuning) de instrucciones ha demostrado mejorar el aprendizaje de Zero-Shot [Wei et al. \(2022\)](#). El método [RLHF \(aprendizaje por refuerzo a partir de retroalimentación humana\)](#) ha sido adoptado para escalar el fine-tuning de instrucciones, donde el modelo se alinea mejor con las preferencias humanas. Este desarrollo reciente potencia modelos como ChatGPT.

Cuando el zero-shot no funciona, se recomienda proporcionar demostraciones o ejemplos en el *prompt*, lo que conduce a la generación de *prompts* de pocos disparos (few-shot).

Few-Shot Prompting

Aunque los modelos de lenguaje de gran escala demuestran capacidades notables con zero-shot, , puede que el modelo se quede corto en tareas más complejas con dicho método. La técnica de *prompts* de few-shot puede usarse para habilitar el aprendizaje en contexto (in-context learning), donde proporcionamos demostraciones en el *prompt* para dirigir al modelo hacia un mejor rendimiento. Las demostraciones sirven como condicionamiento para ejemplos subsiguientes en los que nos gustaría que el modelo genere una respuesta.

Según [Touvron et al. 2023](#) las propiedades de few-shot aparecieron por primera vez cuando los modelos se escalaron a un tamaño suficiente ([Kaplan et al., 2020](#)).

Prompt:

Clasificar los textos como se muestra en los ejemplos.
Indicar solamente la clasificación faltante.

Texto: Hoy el clima está fantástico
Clasificación: Pos
Texto: Los muebles son pequeños.
Clasificación: Neg
Texto: No me gusta tu actitud
Clasificación: Neg
Texto: Ese tiro fue terrible
Clasificación:

Salida:

Neg

Limitaciones del Few-shot Prompting

La técnica estándar de prompts de few-shot funciona bien para muchas tareas, pero aún no es perfecta, especialmente cuando se trata de tareas de

razonamiento más complejas. Vamos a demostrar por qué es este el caso. Recordemos el ejemplo anterior en el que proporcionamos la siguiente tarea:

Los números impares de este grupo suman un número par: 15, 32, 5, 13, 8 2, 7, 1.

A:

Si lo intentamos de nuevo, el modelo produce lo siguiente:

Sí, los números impares de este grupo suman 107, que es un número par.

Esta no es la respuesta correcta, lo que no sólo resalta las limitaciones de estos sistemas sino que también existe la necesidad de una ingeniería rápida más avanzada.

Intentemos agregar algunos ejemplos para ver si unos few-shot mejoran los resultados.

Prompt:

Los números impares de este grupo suman un número par: 4, 8, 9, 15, 12, 2, 1.

A: La respuesta es falsa.

Los números impares de este grupo suman un número par: 17, 10 , 19, 4, 8, 12, 24.

A: La respuesta es Verdadera.

Los números impares de este grupo suman un número par: 16, 11, 14, 4, 8, 1 3, 24.

A: La respuesta es Verdadero.

Los números impares de este grupo suman un número par: 17, 9, 10, 12, 13, 4, 2.

A: La respuesta es falsa.

Los números impares de este grupo suman un número par: 15 , 32, 5, 13, 8

2, 7, 1.

A:

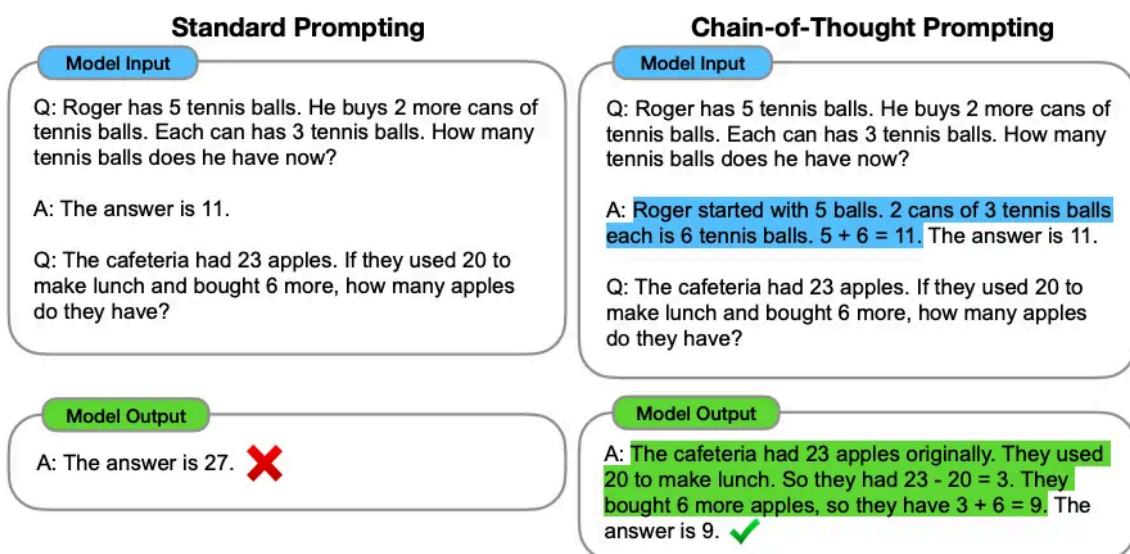
Salida:

La respuesta es Verdadero.

Eso no funcionó. Parece que los few-shot no fueron suficientes para obtener respuestas confiables para este tipo de problema de razonamiento. El ejemplo anterior proporciona información básica sobre la tarea. Si miramos más de cerca, el tipo de tarea que hemos introducido implica algunos pasos de razonamiento más. En otras palabras, podría ser útil dividir el problema en pasos y demostrarlo al modelo. Más recientemente, los prompts de cadena de pensamiento (CoT) se han popularizado para abordar tareas más complejas de razonamiento aritmético, de sentido común y simbólico.

En general, parece que proporcionar ejemplos resulta útil para resolver algunas tareas. Cuando los *prompts* de zero-shot y few-shot no son suficientes, podría significar que todo lo que aprendió el modelo no es suficiente para realizar bien la tarea. A partir de aquí se recomienda empezar a pensar en ajustar sus modelos o experimentar con técnicas de *prompts* más avanzadas.

Chain-of-Thought (CoT) Prompting



Fuente: [Wei et al. \(2022\)](#)

Introducido en Wei et al. (2022), el prompting de cadena de pensamiento (CoT, por sus siglas en inglés) permite capacidades de razonamiento complejas a través de pasos de razonamiento intermedios. Podemos combinarlo con el prompting de few-shot para obtener mejores resultados en tareas más complejas que requieren razonamiento antes de responder.

Prompt:

Los números impares de este grupo suman un número par: 4, 8, 9, 15, 12, 2, 1.

A: Sumar todos los números impares (9, 15, 1) da 25. La respuesta es falsa.

Los números impares de este grupo suman un número par: 17, 10, 19, 4, 8, 12, 24.

A: Sumar todos los números impares (17, 19) da 36. La respuesta es verdadera.

Los números impares de este grupo suman un número par: 16, 11, 14, 4, 8, 13, 24.

A: Sumar todos los números impares (11, 13) da 24. La respuesta es verdadera.

Los números impares en este grupo suman hasta un número par: 17, 9, 10, 12, 13, 4, 2.

A: Sumar todos los números impares (17, 9, 13) da 39. La respuesta es falsa.

Los números impares de este grupo suman a un número par: 15, 32, 5, 13, 82, 7, 1.

A:

Output:

Sumar todos los números impares (15, 5, 13, 7, 1) da 41. La respuesta es falsa.

Podemos ver un resultado perfecto cuando proporcionamos los pasos de razonamiento. De hecho, podemos resolver esta tarea proporcionando incluso

menos ejemplos, es decir, un solo ejemplo parece suficiente:

Prompt:

Los números impares de este grupo suman un número par: 4, 8, 9, 15, 12, 2, 1.

A: Sumar todos los números impares (9, 15, 1) da 25. La respuesta es falsa.

Los números impares de este grupo suman un número par: 15, 32, 5, 13, 8, 2, 7, 1.

A:

Output:

Sumar todos los números impares (15, 5, 13, 7, 1) da 41. La respuesta es falsa.

Tengamos en cuenta que los autores afirman que se trata de una habilidad emergente que surge con modelos de lenguaje suficientemente grandes.

Zero-Shot CoT Prompting

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Fuente: [Kojima et al. \(2022\)](#)

Una idea reciente que ha surgido es la idea de la CoT de cero disparos (Kojima et al. 2022) que básicamente implica agregar "Pensemos paso a paso" al prompt original. Probemos un problema simple y veamos cómo se desempeña el modelo:

Prompt:

Fui al mercado y compré 10 manzanas. Le di 2 manzanas al vecino y 2 al reparador. Luego fui y compré 5 manzanas más y me comí 1. ¿Con cuántas manzanas me quedé?

Output:

11 manzanas

La respuesta es incorrecta. Ahora intentemos con el *prompt* especial:

Prompt:

Fui al mercado y compré 10 manzanas. Le di 2 manzanas al vecino y 2 al reparador. Luego fui y compré 5 manzanas más y me comí 1. ¿Con cuántas manzanas me quedé?

Pensemos paso a paso.

Salida:

Primero, comenzaste con 10 manzanas.
Le diste 2 manzanas al vecino y 2 al reparador, así que te quedaron 6 manzanas.
Luego compraste 5 manzanas más, así que ahora tenías 11 manzanas.
Finalmente, te comiste 1 manzana, así que te quedarías con 10 manzanas.

Es impresionante que este simple *prompt* sea efectivo en esta tarea. Esto es particularmente útil cuando no tenemos demasiados ejemplos para usar en el *prompt*.

ReAct Prompting

Yao et al., 2022 introdujeron un marco llamado ReAct donde los LLM se utilizan para generar rastros de razonamiento y acciones específicas de tareas de manera intercalada.

La generación de rastros de razonamiento permite que el modelo induzca, rastree y actualice planes de acción, e incluso maneje excepciones. El paso de acción permite interactuar y recopilar información de fuentes externas, como bases de conocimiento o entornos.

El marco ReAct puede permitir a los LLM interactuar con herramientas externas para recuperar información adicional que conduzca a respuestas más confiables y objetivas.

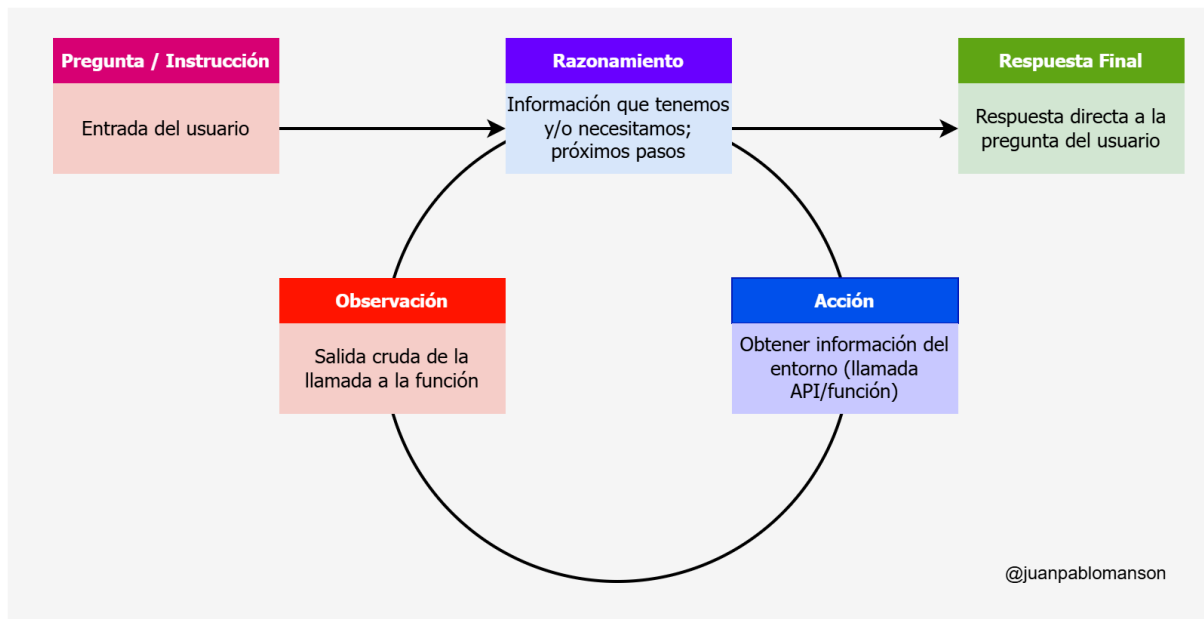
Los resultados muestran que ReAct puede superar varias líneas de base de última generación en tareas de lenguaje y toma de decisiones. ReAct también conduce a una mejor interpretabilidad humana y confiabilidad de los LLM. En general, los autores encontraron que el mejor enfoque utiliza ReAct combinado con cadena de pensamiento (CoT) que permite el uso tanto del conocimiento interno como de la información externa obtenida durante el razonamiento.

¿Cómo funciona ReAct?

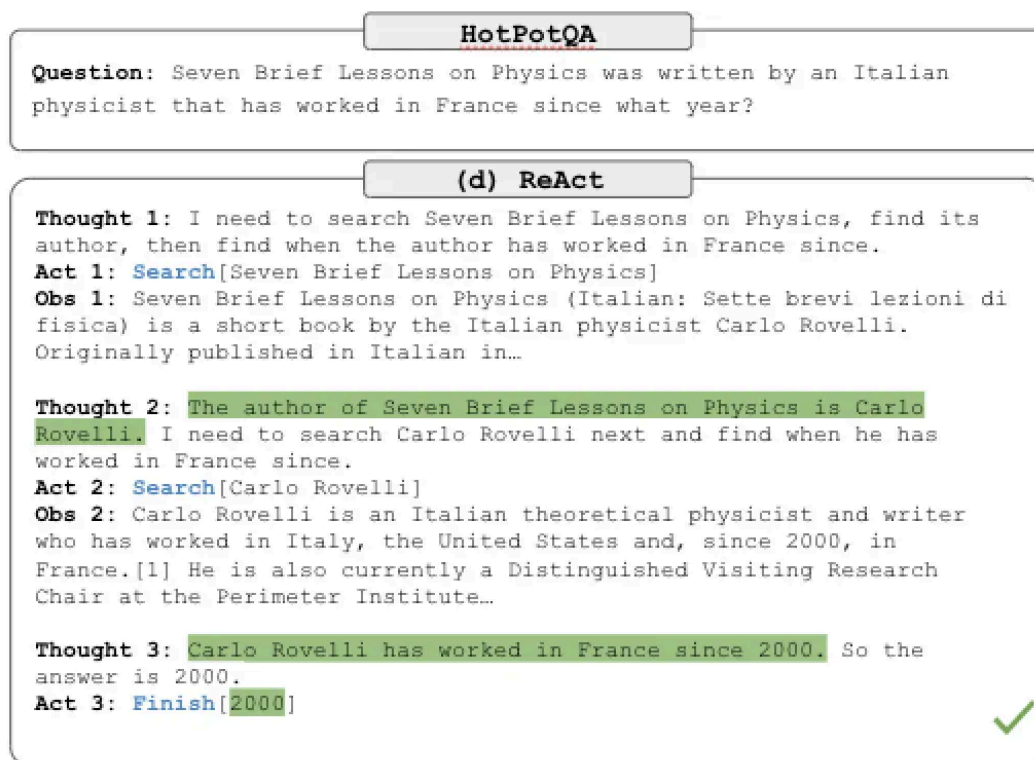
ReAct se inspira en las sinergias entre "actuar" y "razonar" que permiten a los humanos aprender nuevas tareas y tomar decisiones o razonar.

Los prompts de cadena de pensamiento (CoT) han demostrado las capacidades de los LLM para realizar rastreos de razonamiento para generar respuestas a preguntas que involucran razonamiento aritmético y de sentido común, entre otras tareas (Wei et al., 2022). Pero su falta de acceso al mundo exterior o su incapacidad para actualizar sus conocimientos puede provocar problemas como alucinaciones reales y propagación de errores.

ReAct es un paradigma general que combina el razonamiento y la acción con los LLM. ReAct solicita a los LLM que generen líneas de razonamiento verbal y acciones para una tarea. Esto permite que el sistema realice un razonamiento dinámico para crear, mantener y ajustar planes de acción y al mismo tiempo permite la interacción con entornos externos (por ejemplo, Wikipedia) para incorporar información adicional al razonamiento.



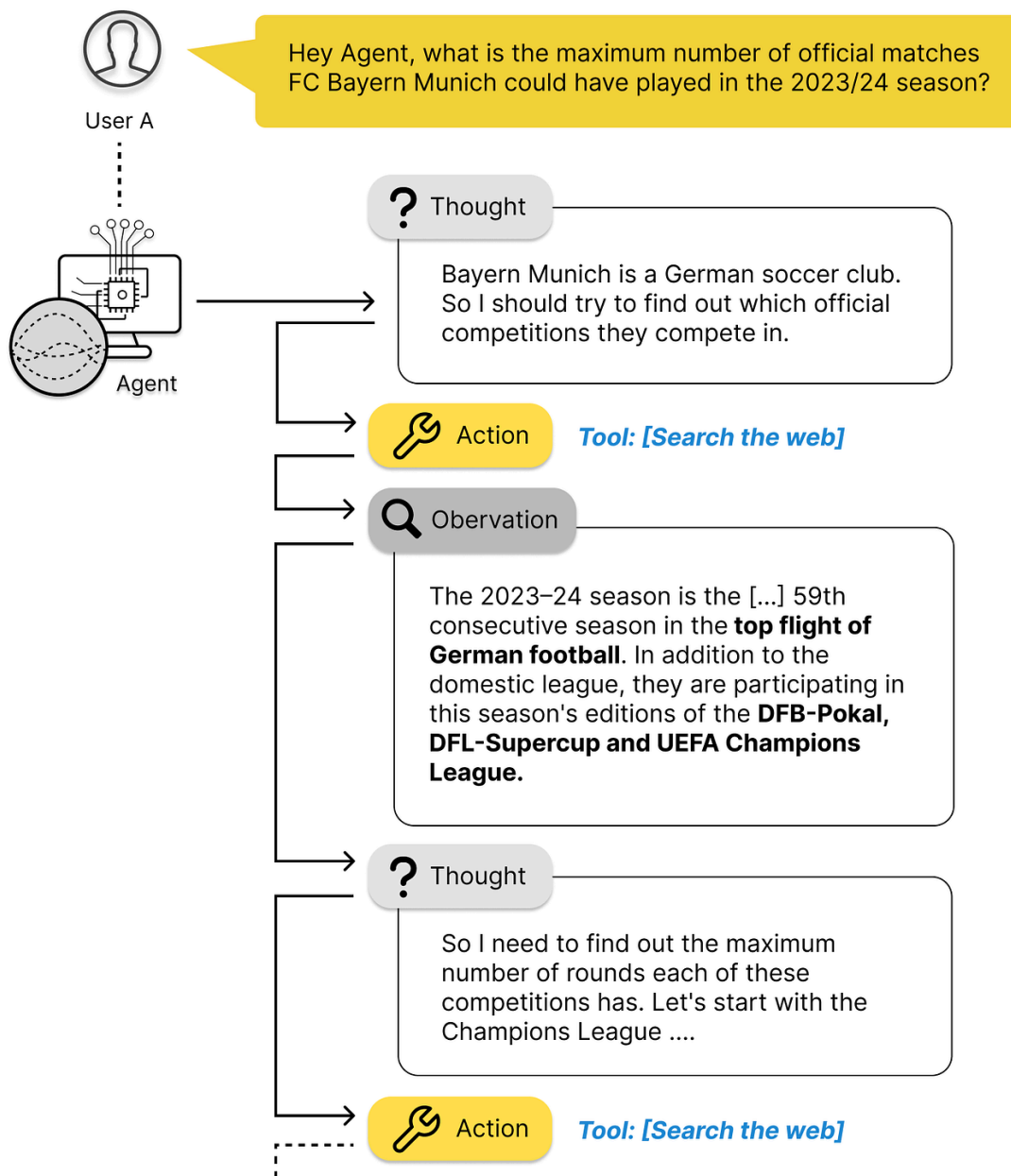
La siguiente figura muestra un ejemplo de ReAct y los diferentes pasos necesarios para responder preguntas.



Podemos ver que el modelo genera trayectorias de resolución de tareas (Thought, Act). Obs corresponde a la observación del entorno con el que se interactúa (por ejemplo, motor de búsqueda). En esencia, ReAct puede

recuperar información para respaldar el razonamiento, mientras que el razonamiento ayuda a determinar qué recuperar a continuación.

Aquí vemos otro gráfico que nos ayuda a entender el flujo de ReAct:



Fuente: <https://towardsdatascience.com/navigating-the-world-of-llm-agents-a-beginners-guide-3b8d499db7a9>



El resultado de llamar a las herramientas, es clave para construir la respuesta del usuario. Un error en la llamada o recuperar resultados que no tienen que ver con la consulta del usuario, pueden hacer perder el hilo de la conversación y no llegar al resultado esperado, incluso seguir iterando de forma indefinida. Es por eso que suele ponerse un límite de iteraciones para evitar estas situaciones.

Podemos resumir de ReAct:

- CoT sufre de alucinaciones de hechos.
- La restricción estructural de ReAct reduce su flexibilidad para formular pasos de razonamiento.
- ReAct depende mucho de la información que recupera; Los resultados de búsqueda no informativos descarrilan el razonamiento del modelo y provocan dificultades para recuperar y reformular pensamientos.

Más información

ReAct: Synergizing Reasoning and Acting in Language Models

<https://react-lm.github.io/>

<https://docs.google.com/presentation/d/1xIZaZHkARAp38TAZsoxIctPTuNW3PwX44oq7kGEWEyk/edit?usp=sharing>